Recursion

COP 3502C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

Scenario

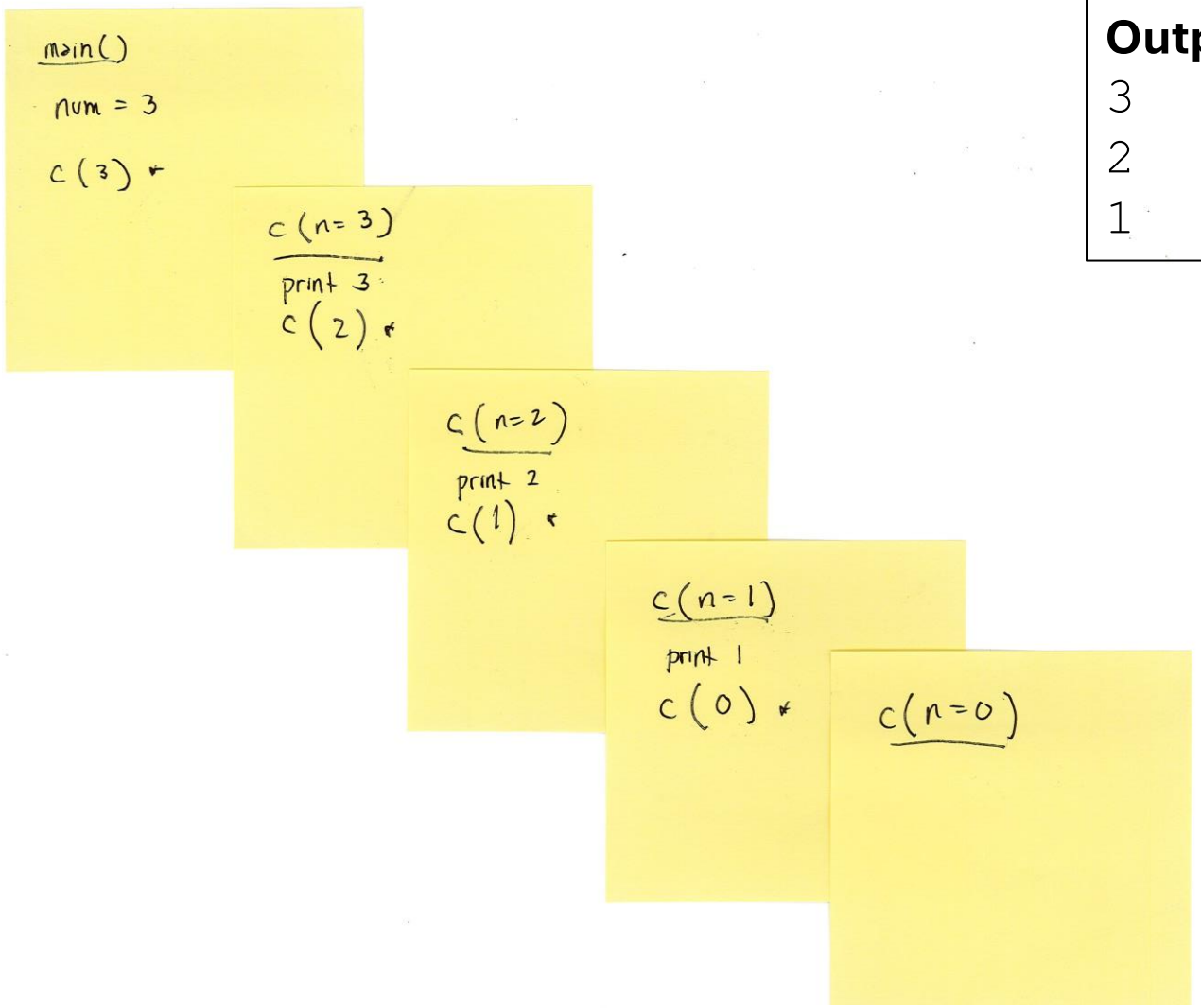
- Write a program asks the user to enter a whole number **`N`**

- Perform a countdown from **`N`** to **`1`**

Sample Run

```
Enter Number: 3
3
2
1
```

```
1  void c( int n ) {
2      if ( n == 0 ) {
3          return;
4      }
5      printf("%d\n", n);
6      c(n-1);
7  }
8  int main (void) {
9      int num = 3;
10     c(num);
11     return 0;
12 }
```



Output

```
3
2
1
```

A single Post-It note represents a single call frame

Notes /1

- For a **`void`** function, there is an implicit **`return`** statement at the end of the function's definition

- The local variables of a function are not global variables; therefore, they are not shared

Notes /2

- Each function call has its own copy of all the local variables

- Even if they have the same name, they are different entities (think of two different persons with the same name)

Recursive Functions (Parts)

Recursive Case

Reduction Step

Base Case

Stopping Condition

Recursive Functions

- It can be a **`void`** or a non-**`void`** function
- How can I **break this big problem** into smaller subproblems?
- Once the problem is **small enough**, how do I solve it?
- How do I **combine** the results?

Scenario

- Write a program asks the user to enter a whole number **`N`**

- Afterward, print the factorial of **`N`**

Sample Run

```
Enter Number: 5
5! = 120
```

The `main()` Function

```
int main(void) {
    int N;

    printf("Enter Number: ");
    scanf("%d", &N);

    // TODO: call function for factorial

    return 0;
}
```

Solution /1 – Version 1

```
int factorial_v1(int n) {
    int result = 1;
    for(int i = n; i >= 1; i--) {
        result = result * i;
    }
    return result;
}
```

What is the overall running time of this function?

Solution /2 – Version 2

```
int factorial_v2(int n) {
    int result = 1;
    while(n >= 1) {
        result = result * n;
        n--;
    }
    return result;
}
```

What is the overall running time of this function?

Notes

- The two versions we saw were examples of **iterative** solutions

- Used **loops** where blocks of codes were **repeated**

Another Way of Thinking /1

- Define a problem in terms of a simpler version of **itself**

- A problem-solving strategy we refer to as **recursion**

Another Way of Thinking /2

- Define a simple function that can compute **`N!`**

- Look closely at the **definition** of factorial

Definition of Factorial

$$n! = \begin{cases} n * (n-1)! & , \text{ if } n > 1 \\ 1 & , \text{ if } n = 1 \end{cases}$$

Solution /3 – Version 3

```
int factorial_v3(int n) {
    if(n > 1) {
        return n * factorial_v3(n-1);
    }
    else {
        return 1;
    }
}
```

Solution /4 – Version 4

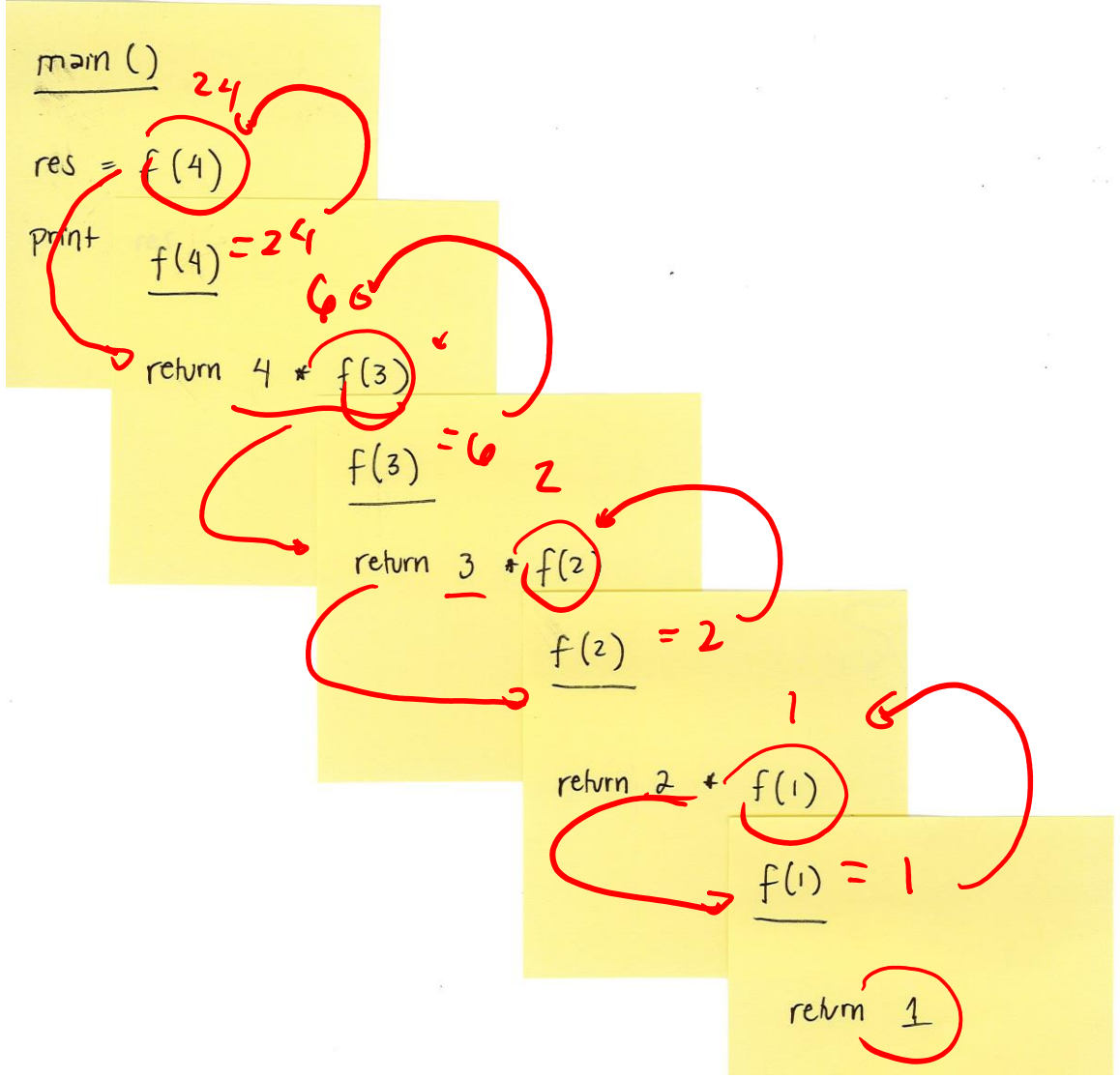
```
int factorial_v4(int n) {
    if(n <= 1) {
        return 1;
    }

    return n * factorial_v4(n-1);
}
```

Equivalent Logic

```
int factorial_v3(int n) {
    if(n > 1) {
        return n * factorial_v3(n-1);
    }
    else {
        return 1;
    }
}
```

```
int factorial_v4(int n) {
    if(n <= 1) {
        return 1;
    }

    return n * factorial_v4(n-1);
}
```

```
① int f(int n) {
②     if (n <= 1) {
⑤         return 1;
④     }

⑤     return n * f(n-1);
⑥ }

⑦ int main(void) {
⑧     int res = f(4);
⑨     printf("%d", res);

⑩     return 0;
⑪ }
```



Output

24

A single Post-It note represents a single call frame

Notes

- When a function is non-**`void`**, it promises that it returns a single value with the specified data type (i.e., function's return data type)

- When tracing a **`return`** statement, there must only be a single value for the function to return to its caller

- Otherwise, the expression must be evaluated first such that it simplifies to a single value (e.g., the expression **`n * f(n-1)`** must be reduced to a single value first before we can return)

Code Tracing /1

```
void f1(int n) {
    if(n < 1) {
        printf("Done!\n");
        return;
    }

    printf("%d\n", n);
    f1(n-1);
}
```

Demonstrated in class via Post-It notes

Code Tracing /2

```
void f2(int n) {
    if(n < 1) {
        printf("Done!\n");
        return;
    }

    f2(n-1);
    printf("%d\n", n);
}
```

Demonstrated in class via Post-It notes

Code Tracing /3

```
void f3(int n) {
    printf("%d\n", n);
    f3(n-1);

    if(n < 1) {
        printf("Done!\n");
        return;
    }
}
```

Notes /1

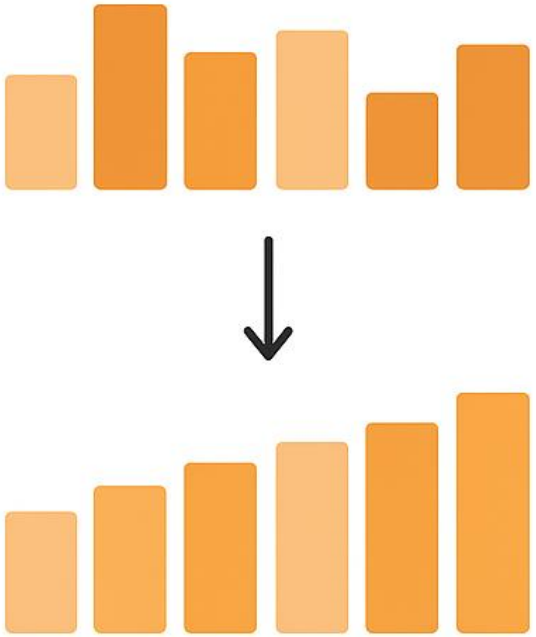
- The first two codes illustrated two **types of recursion**
- **Tail recursion** is where the recursive call is the last operation
- **Head recursion** is where the recursive call happens before any other operations
- There are other types such as indirect recursion

Notes /2

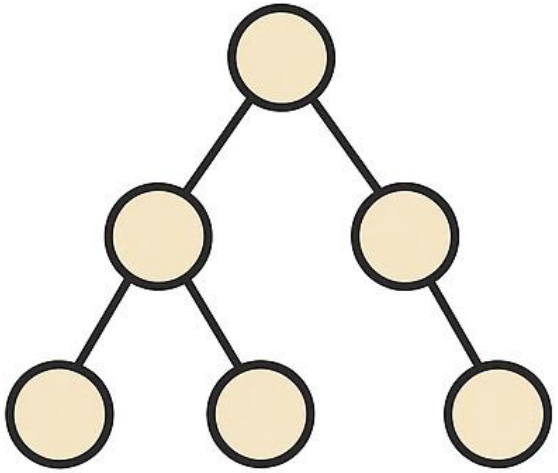
- For the third code, we ended with a stack overflow
- Essentially, you will see a **Segmentation Fault***
- Comment out the printing to “speed up” the program
- The order here matters; so be careful with your design!

OpenAI. (2026). *Matryoshka dolls in sequence* [AI-generated image]. ChatGPT. Retrieved from https://chat.openai.com/

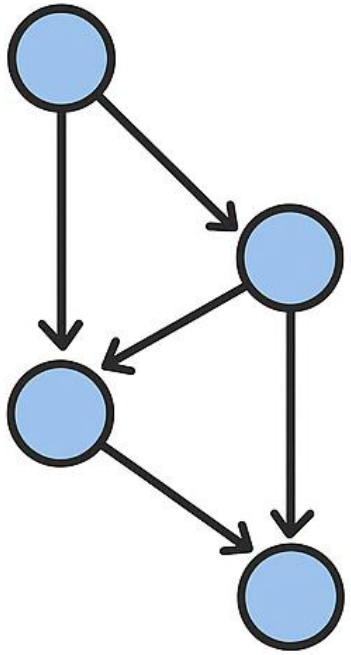
Where we see them in Computer Science?



Sorting



Tree Traversal



Graph Traversal

OpenAI. (2025). *A simple drawing of CS concepts* [AI-generated image]. ChatGPT. Retrieved from https://chat.openai.com/

Discussion /1

- Allows for simple, **elegant**, and **readable** solutions (i.e., *natural*)

- Compare the iterative and recursive versions of our solution

Discussion /2

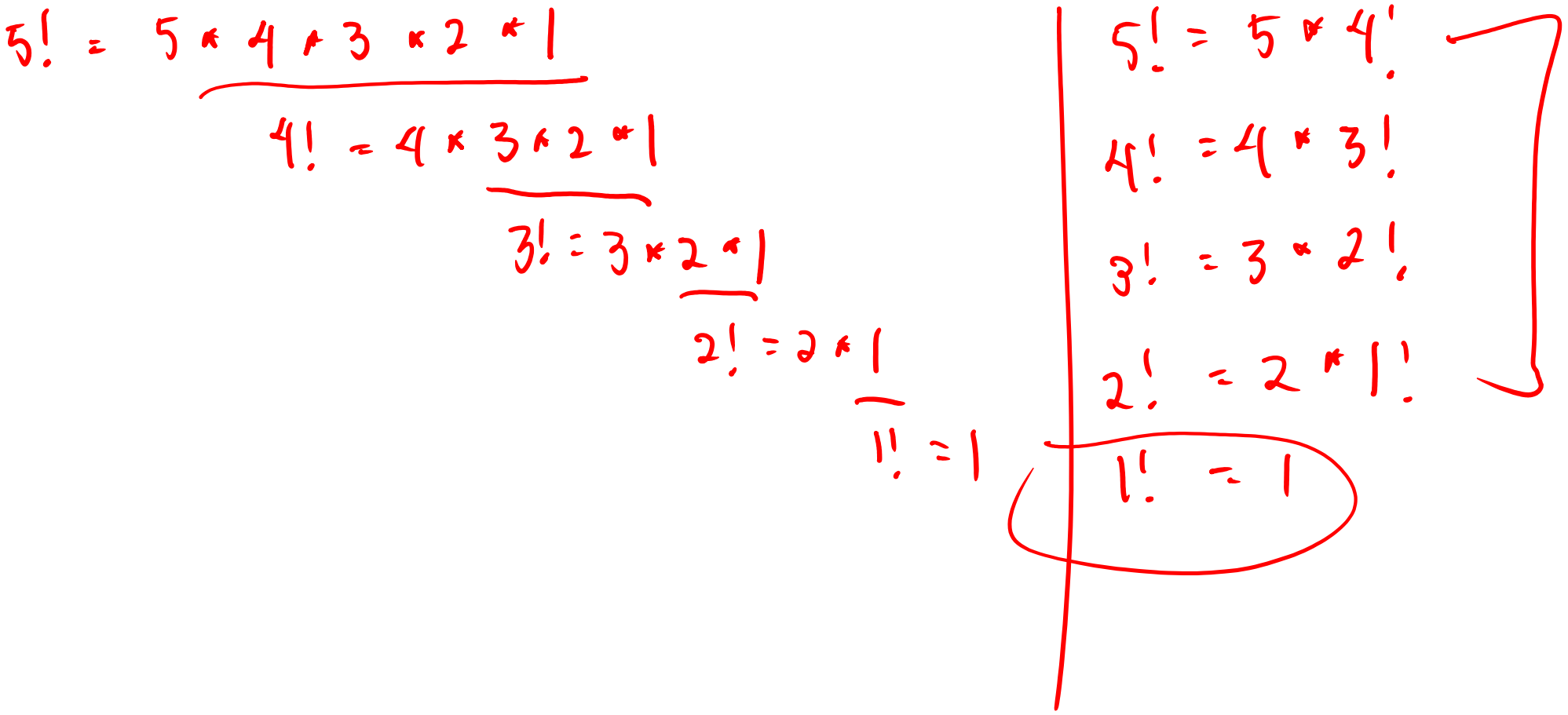
- Can you think of a potential issue?
- Imagine it being called without a stopping condition?
- You'll end up in a situation called a **stack overflow**
- Somewhat "similar" to an infinite loop

Discussion /3

Tendency to be **less efficient** than its iterative counterpart due to **overhead** involved

Example

Write a function `factorial(n)` that returns `n!` (as in `n` factorial)



Practice

Write a recursive function `power(a, n)` which returns a^n

base exponent

2^n

$2^5 = 2 * 2 * 2 * 2 * 2$

$2^4 = 2 * 2 * 2 * 2$

$2^3 = 2 * 2 * 2$

$2^2 = 2 * 2$

$2^1 = 2$

$2^5 = 2 * 2^4$

$2^4 = 2 * 2^3$

$2^3 = 2 * 2^2$

$2^2 = 2 * 2^1$

$2^1 = 2 * 2^0$

$2^0 = 1$

$$a^n = \begin{cases} a * a^{n-1} & , \text{if } n > 1 \\ a & , \text{if } n = 1 \\ 1 & , \text{if } n = 0 \end{cases}$$

```
int power(int a, int n) {

    // base case
    if(n == 1) {
        return a;
    }

    // base case
    if(n == 0) {
        return 1;
    }

    // recursive case
    return a * power(a, n-1);
}
```

Final Notes /1

- Familiarize the **nature** of recursion
- The problem can be broken into smaller subproblems
- Essentially, solving a *smaller version* of the problem
- Need to have **base case(s)** (i.e., problem is small enough)

Final Notes /2

- Where in the function you put the recursive call determines the type of recursion you are designing

- Recursion can also be **indirect** (`A` calls `B` which calls `A`)

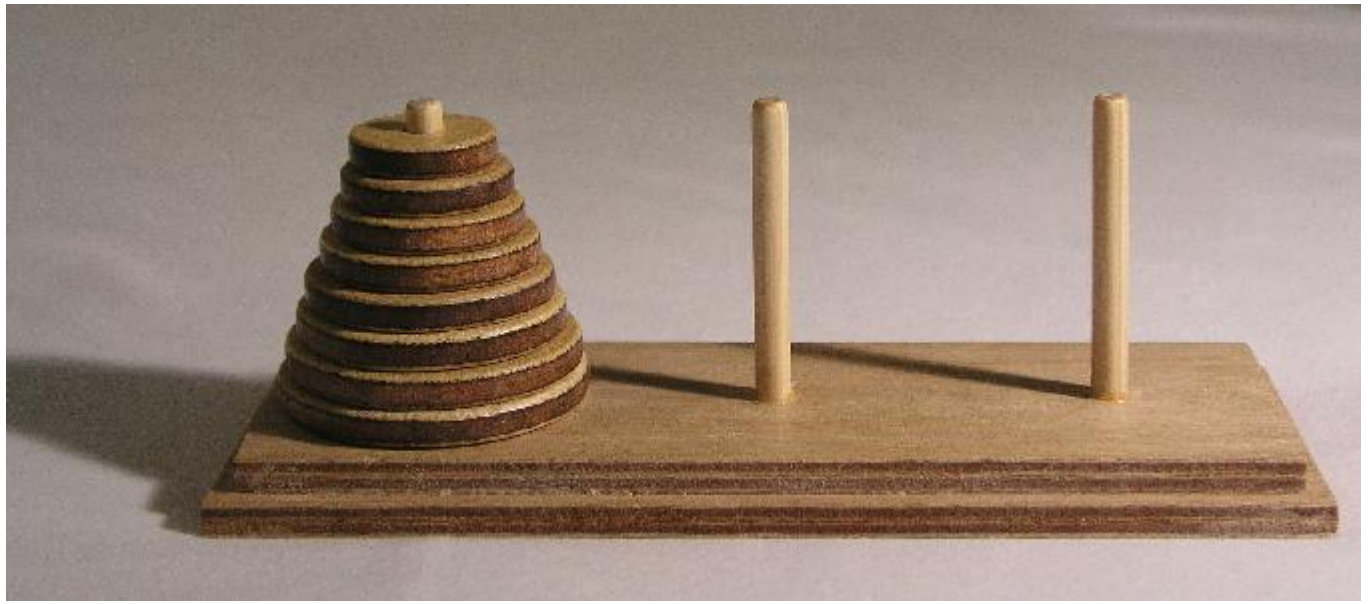
Challenge /1

Write a recursive function `fib(n)` that returns the `n`-th term in the **Fibonacci sequence**. Assume that the first two terms are 0 and 1.

$$\text{fib}\,(n) = \begin{cases} \text{fib}\,(n-2) + \text{fib}\,(n-1) & , \text{ if } n \geq 2 \\ 1 & , \text{ if } n = 1 \\ 0 & , \text{ if } n = 0 \end{cases}$$

Challenge /2

Solve the **Tower of Hanoi** problem involving `N` disks and 3 poles



Source: https://upload.wikimedia.org/wikipedia/commons/0/07/Tower_of_Hanoi.jpeg

Questions?